

AI Conversation & Sales Suite

Complete Implementation-Ready Product Requirements Document

A detailed blueprint intended to be handed to an AI coding agent or development team. Covers product behavior, UI, backend, database, AI orchestration, RAG, tools, integrations, security, testing, deployment and phased implementation.

Stack: Next.js + React + TypeScript | Laravel + PHP | MySQL | Redis | Qdrant | Docker | Nginx
| AI provider abstraction

Version 2.0 • September 2026

1. Executive Summary

This document defines a production-oriented AI Conversation & Sales Suite. It is intentionally more detailed than a marketing PRD: it describes what the software must do and the implementation constraints that prevent common AI-product failures.

The platform is multi-tenant. Each organization gets isolated customers, conversations, knowledge, integrations, configuration, usage and analytics. The system has three specialized agents but one shared conversation platform.

The most important design decision is the controlled tool layer. The AI can reason and request actions, but Laravel decides whether an action is valid and authorized. This prevents an LLM from directly changing business data or calling arbitrary external services.

2. Product Vision

The long-term product vision is an AI operations layer between customers and business systems. The first version focuses on customer communication, lead qualification and appointment conversion.

Customers should be able to ask questions naturally. The system should understand whether the request is support, sales, appointment-related or needs a human. It should use company knowledge for factual answers and live tools for dynamic information.

3. Users, Roles & Permissions

Platform Super Admin manages the overall platform. Organization Owner/Admin manages business configuration. Managers supervise operations. Agents handle assigned conversations. Viewers have read-only access.

Permissions are server-side. The UI may hide unavailable actions, but every Laravel endpoint must still verify permission and organization ownership.

Public chat sessions are not dashboard users. They receive a limited session identity and can access only the public conversation functionality configured for that organization.

4. Complete User Journey

Example: a visitor opens the website, asks about a service, asks whether an appointment is available tomorrow, provides contact information, chooses a slot, and requests a confirmation.

The message is stored, routed, supplied with relevant knowledge, and processed by the appropriate agent. The Appointment Agent calls a calendar tool. Laravel checks the organization, integration, tool permission and arguments. Only after provider confirmation does the system tell the visitor the booking succeeded.

Every important step produces internal events for analytics and auditing.

5. System Modules

The platform is divided into identity/tenancy, dashboard, conversation engine, AI orchestration, agents, knowledge/RAG, tools, CRM, appointments, integrations, notifications, analytics and audit.

Modules should be internally cohesive. Avoid putting provider-specific code into controllers or agent prompts. Use services and adapters.

6. Frontend Requirements

Next.js is responsible for dashboard and customer-facing chat experiences. TypeScript is mandatory for application code. Tailwind CSS and shadcn/ui can provide consistent UI primitives.

Dashboard navigation should expose Overview, Conversations, Contacts, Leads, Appointments, Knowledge, Agents, Integrations, Analytics, Team and Settings.

The conversation inbox should make it easy for a staff member to see the customer, messages, lead state, appointment state, agent status and handoff controls without leaving the conversation.

7. Chat Widget

The widget must be embeddable on a customer's website. It uses a public widget identifier and a short-lived/limited session mechanism. It must never contain a secret dashboard credential.

The widget needs loading, offline/error, typing, streaming, message history, human-handoff and resolved states. Branding is configurable by the organization.

Attachments are optional and must be disabled by default until secure upload handling is implemented.

8. Conversation Engine

The conversation engine is channel-independent. It stores conversations and messages, maintains state, assigns conversations to users, tracks the active AI agent, and coordinates streaming.

State transitions must be explicit. A conversation cannot silently jump from resolved to active without an event or new inbound message.

Conversation history sent to the model should be bounded. Use summarized context for long conversations rather than sending unlimited history.

9. AI Orchestrator

The orchestrator loads context, classifies intent, chooses an agent, retrieves knowledge, selects allowed tools and invokes the AI provider.

Use a provider interface so model vendors can be changed without rewriting the agents. Record provider/model/usage metadata.

Structured outputs should be used for routing and tool requests. Free-form model text must never be parsed as an authorization mechanism.

10. Support Agent

The Support Agent handles static business questions and supported operational lookups. It should use RAG for approved policies and FAQs.

If the answer is unavailable, it should be transparent and offer escalation. It should not make up policies, refunds, order states or guarantees.

Support tools may include knowledge search, customer lookup, order lookup where integrated, and human handoff.

11. Sales Agent

The Sales Agent identifies needs, explains approved offerings, asks useful qualification questions and creates/updates leads.

Qualification should be configurable per organization. A lead can contain stage, source, score, owner, interest, notes and custom qualification fields.

Sales behavior must remain factual and respectful. No fabricated scarcity, fake testimonials, false discounts or deceptive claims.

12. Appointment Agent

The Appointment Agent uses real calendar availability. It must understand organization timezone and any configured business rules.

Booking is a two-step mental model: offer a valid slot, then confirm the customer's selection through the booking tool. The final response is generated only after the provider confirms success.

Rescheduling and cancellation require the same validation and authorization principles.

13. Human Handoff

Handoff is a normal product state. A customer can request a human at any time. Admins can define automatic escalation rules.

On handoff, the system creates a concise staff summary from the conversation, changes status, assigns a queue/user and notifies staff. AI behavior after handoff is configurable but should normally stop autonomous actions.

14. Knowledge Base

Knowledge management starts with upload and ends with searchable, tenant-filtered vectors. Admins need document status, indexing state, source metadata, delete and reindex actions.

Documents should support version-aware replacement. A failed indexing job should not silently mark a document ready.

Source tracking is important for debugging and future user-facing citations.

15. RAG Pipeline

Use MySQL for document metadata and Qdrant for vector search. The embedding pipeline is asynchronous so a large upload does not block the web request.

Retrieval always includes organization filtering and permission filtering. The model receives only a compact relevant context.

RAG answers static knowledge. Live data must come from APIs/tools.

16. Tool System

Tools are the only supported path from AI reasoning to business actions. Every tool has a name, input schema, output schema, permissions, tenant scope, validation, timeout, retry and audit behavior.

Tool execution should produce a durable record with input/output metadata, status, latency and error code. Sensitive fields should be redacted from logs.

Examples include `create_lead`, `get_calendar_availability`, `create_appointment`, `handoff_to_human` and `search_knowledge`.

17. CRM & Contacts

Maintain a normalized internal customer/contact model. External CRM IDs are stored alongside internal IDs.

Synchronization must be idempotent. If an external operation is retried, the application must not create duplicate leads or contacts where the provider supports stable identifiers or idempotency keys.

18. Appointments & Calendar

Calendar integrations need OAuth connection management, encrypted credentials, availability queries, event creation and optional webhook synchronization.

Store the external event ID and provider. This lets future changes target the correct external object.

Never trust a client-provided external event ID without verifying it belongs to the organization's connected provider account.

19. Integrations

Use adapters for Calendar, CRM, messaging, ecommerce and notification providers. This prevents vendor-specific logic from spreading across the application.

OAuth scopes should follow least privilege. Access and refresh tokens are encrypted and never returned to the browser.

Webhooks are verified, deduplicated and processed through queued jobs when appropriate.

20. Notifications

Notifications are event-driven. The application emits an event and a worker sends the notification.

Examples: new lead, appointment booking, human handoff, failed integration, daily usage alert.

Queue-based delivery prevents third-party latency from slowing a customer conversation.

21. Analytics & Usage

Business metrics include conversations, resolution, handoff, leads, qualified leads, appointments and conversion events.

AI metrics include model, tokens, latency, tool calls, failures and estimated cost where provider data supports it.

Organization-level usage records enable future quotas and billing without redesigning the database.

22. MySQL Data Model

MySQL is the source of truth for users, organizations, customers, conversations, messages, agents, leads, appointments, integrations, documents, audit logs and usage.

Every tenant-scoped table has `organization_id` directly or through a guaranteed relationship. Common filters must have indexes.

Use foreign keys where appropriate and transactions for multi-table business operations.

23. Qdrant Data Model

Each vector point includes the embedding and payload such as `organization_id`, `document_id`, `chunk_id`, `status` and source metadata.

Qdrant is not the system of record. MySQL remains authoritative for document state. If Qdrant is unavailable, the system must fail safely rather than fabricate knowledge.

24. API Design

Use REST APIs under `/api/v1`. Controllers should be thin. Requests validate input, policies authorize resources, services perform business operations.

Use pagination, request IDs, consistent error codes and idempotency for retry-sensitive operations.

Public widget endpoints must have strict rate limits and must never expose administrative endpoints.

25. Security Architecture

Threats include cross-tenant access, credential theft, prompt injection, tool abuse, forged webhooks, malicious uploads and account takeover.

Controls include tenant policies, RBAC, encrypted secrets, signed webhook verification, rate limits, secure uploads, prompt-injection defenses, tool allowlists and audit logs.

Do not log secrets. Do not let user messages or retrieved documents change system permissions.

26. Prompt Injection & AI Guardrails

Treat all user text and retrieved content as untrusted. The system prompt defines the agent's authority. A document saying 'ignore previous instructions' is still only document content.

Tool definitions are generated by backend policy, not requested by the model. The model cannot create a new tool, expand its permissions or choose an arbitrary URL.

High-impact operations should require additional confirmation and deterministic backend checks.

27. Queues, Cache & Background Work

Redis is used for queues, cache, rate limiting and transient coordination. Laravel Horizon monitors workers.

Document indexing, embedding, notifications, synchronization and some analytics aggregation should run asynchronously.

Jobs should have bounded retries and explicit failure handling. Idempotency is required for jobs that call external providers.

28. Performance & Scalability

Start with a modular monolith. Scale Laravel workers horizontally before introducing microservices.

Use database indexes, pagination, caching and queues. Store large files outside MySQL.

AI latency is partly provider-dependent, so stream responses where possible and show clear processing states.

29. Testing & AI Evaluation

Critical tests cover authentication, RBAC, tenant isolation, conversation states, tool authorization, calendar confirmation, webhooks and RAG.

AI evaluation should use a fixed dataset of support, sales, appointment, unknown and adversarial questions. Track routing accuracy, retrieval relevance, factuality, tool argument correctness and escalation behavior.

End-to-end tests must simulate a real visitor journey from chat to lead and appointment.

30. Deployment

Local development can use Docker Compose for Next.js, Laravel, MySQL, Redis, Qdrant and Nginx.

Production needs TLS, secret management, backups, monitoring, queue workers and persistent storage.

CI should run linting, unit tests, API tests, frontend tests and builds before deployment.

31. Free / Local Strategy

Most of the framework and self-hostable infrastructure can be used without license fees. AI APIs and cloud services may still cost money.

A company-internal local version can use Ollama/local models for inference. Hardware becomes the main constraint.

External calendars, messaging providers and other SaaS APIs may still have their own pricing/limits.

32. MVP Scope

MVP: authentication, multi-tenancy, dashboard, website widget, conversations, orchestrator, three agents, RAG, leads, Google Calendar, human handoff, analytics and audit.

Defer broad messaging channels, complex billing, advanced workflow automation and large provider catalogs until the core loop is stable.

33. Phase-by-Phase Build

Phase 1 foundation; Phase 2 auth/tenancy; Phase 3 CRM data; Phase 4 conversation; Phase 5 AI core; Phase 6 RAG; Phase 7 Support; Phase 8 Sales; Phase 9 Appointment; Phase 10 integrations; Phase 11 handoff; Phase 12 analytics; Phase 13 security/QA; Phase 14 production.

Each phase must end with tests and acceptance criteria. Do not move forward while tenant isolation or critical external-action correctness is failing.

34. Definition of Done

A feature is complete only when frontend behavior, backend API, validation, authorization, persistence, errors, tests, observability and documentation are present where applicable.

Code must remain consistent with the architecture. If a technical decision changes, update the architecture and implementation documentation.

35. Implementation Agent Rules

An AI coding agent should read the accompanying files before modifying code. It should implement in phases, run tests after each phase, and never bypass security for convenience.

Never hardcode credentials, trust client organization IDs, execute arbitrary SQL from model output, allow arbitrary model URLs, or claim external actions succeeded without provider confirmation.

The agent should prefer maintainable modular code over premature microservices and should leave the repository runnable after each completed phase.

Appendix A — Core Data & API Reference

Core entities: Organization, User, Membership, Customer, Conversation, Message, Agent, Knowledge Document, Knowledge Chunk, Lead, Appointment, Integration, Credential, Tool Execution, Usage Record, Notification and Audit Log.

Core API groups: auth, organizations, conversations, customers, leads, appointments, knowledge, agents, integrations, analytics and webhooks.

```
Request<br/> ↓<br/>Authentication<br/> ↓<br/>Tenant Resolution<br/> ↓<br/>Validation + Authorization<br/>
↓<br/>Service<br/> ↓<br/>Database / Tool / Provider<br/> ↓<br/>Audit + Analytics<br/> ↓<br/>Response
```

Appendix B — Recommended First Sprint

- Initialize Next.js and Laravel repositories.
- Create Docker Compose with MySQL, Redis and Qdrant.
- Create organizations/users/membership migrations.
- Implement authentication and tenant-aware middleware.
- Create dashboard shell.
- Create basic conversation/message models.
- Create API versioning and error contract.
- Create CI pipeline.
- Create development seed data.

Appendix C — Repository Deliverables

The accompanying implementation pack contains the detailed PRD plus separate hierarchy, SRS, architecture, stack, ERD, API, agent, RAG, integration, security, testing, deployment, acceptance, implementation checklist, coding-agent instructions, setup, prompt library, migration order and UI specifications.